



rewrite in report format all the REST API implemented.

The attached `rest_api_summary.md` already has a strong structured overview; below is a rewritten version in a more formal “report” format, keeping your technical level in mind and organizing all implemented REST APIs into sections and subsections.^[1]

1. Introduction

This report describes the REST APIs implemented by the `libcloud.rest` service. The service is a FastAPI-based wrapper around Apache Libcloud drivers for Nutanix and AWS, exposing a unified, stateless interface for compute, storage, networking, and auxiliary cloud operations.^[1]

The current implementation provides 52 REST endpoints grouped into 7 functional categories:

- Authentication and identity
- Provider and capability discovery
- Connection bootstrap and capability introspection
- Compute services (locations, images, sizes, nodes)
- Storage services (volumes and snapshots)
- Network services (floating IPs, key pairs, security groups, firewalls)
- Miscellaneous / operational utilities (ping, health, metadata, async job tracking)^[1]

Every stateful interaction with cloud providers is encapsulated by Libcloud drivers; the API server itself does not persist connection objects and instead requires the client to supply all necessary connection parameters on each request.^[1]

2. Authentication APIs

2.1 Purpose and Scope

The authentication APIs under `/v1/auth` manage token lifecycle and identity-related queries for clients of the `libcloud.rest` service. They implement short-lived JWT access tokens (HS256) and longer-lived opaque refresh tokens, and also expose an admin-only token introspection endpoint.^[1]

2.2 Endpoints

- `POST /v1/auth/login`
Validates credentials and issues a JWT access token (HS256, 15-minute expiry) plus an opaque refresh token with an 8-hour lifetime. ^[1]
- `POST /v1/auth/refresh`
Renews an expired or soon-to-expire access token using a valid refresh token, preserving identity and scope while updating token validity. ^[1]
- `POST /v1/auth/logout`
Revokes the JTI of the current access token and optionally revokes the associated refresh token, effectively ending the session. ^[1]
- `GET /v1/auth/me`
Returns identity metadata for the current token, including username, tenant, scope, allowed providers, and session identifier. ^[1]
- `POST /v1/auth/token/introspect`
Admin-only endpoint (RFC 7662-style) that decodes and validates an arbitrary token, returning its status and claims. ^[1]

3. Provider Discovery API

3.1 Purpose and Scope

The provider discovery API under `/v1/providers` exposes a static description of supported cloud providers and the classes of operations available for each. It is intended for client-side capability discovery and UI/API auto-configuration. ^[1]

3.2 Endpoint

- `GET /v1/providers`
Returns the list of supported providers (currently `aws` and `nutanix`) along with their supported operation groups (compute, volumes, snapshots, key pairs, security features, etc.). ^[1]

4. Connection API

4.1 Purpose and Scope

The connection API under `/v1/connections` allows clients to submit connection parameters and receive details about the constructed Libcloud driver and its capabilities. The server deliberately avoids persisting these connection objects; each request is fully self-contained. ^[1]

4.2 Endpoint

- `POST /v1/connections:test`

Builds a driver instance from client-supplied connection data and reports capabilities such as support for volumes, snapshots, key pairs, `wait_until_running`, and authentication modes. The connection object is discarded after the response, enforcing statelessness on the server side.^[1]

5. Compute APIs

The compute APIs under `/v1/compute` provide location, image, size discovery and full VM lifecycle management across AWS and Nutanix. All endpoints require a Bearer token and a connection payload; every handler begins by calling `policy_engine.authorize_connection()` to enforce authorization.^[1]

5.1 Locations, Images, and Sizes

These endpoints focus on discovery and image management.^[1]

- `GET /v1/compute/locations`
Returns AWS regions and availability zones using Libcloud's `list_locations()` and Nutanix clusters via `ex_list_clusters()`.^[1]
- `GET /v1/compute/images`
Lists available images. For AWS, it defaults to filtering for images matching “Ubuntu” but allows overriding via a name filter.^[1]
- `POST /v1/compute/images`
Nutanix-only endpoint that creates images via `ex_create_image_from_url()` (URL-based image import) or `create_image()` derived from an existing VM. The operation can be executed asynchronously.^[1]
- `DELETE /v1/compute/images/{image_id}`
Deletes an image on either AWS or Nutanix, identified by `image_id`.^[1]
- `GET /v1/compute/sizes`
Returns instance types and sizes using Libcloud's `list_sizes()` for the given provider.^[1]

5.2 Nodes / VM Lifecycle

These endpoints implement full lifecycle operations for virtual machines (nodes).^[1]

- `GET /v1/compute/nodes`
Lists all VMs visible in the connection's scope, including basic metadata and state.^[1]
- `GET /v1/compute/nodes/{node_id}`
Returns detailed information for a specific node identified by `node_id`.^[1]
- `POST /v1/compute/nodes`
Creates a VM with rich parameters: size, image, location, authentication configuration,

network setup, tags, and provider-specific options. The endpoint supports asynchronous creation, returning job metadata when not executed synchronously. ^[1]

- `PATCH /v1/compute/nodes/{node_id}`
Updates VM properties. On Nutanix this includes name, description, and memory settings; on AWS it supports resize operations and tag updates. ^[1]
- `POST /v1/compute/nodes/{node_id}:start`
Starts a stopped VM. ^[1]
- `POST /v1/compute/nodes/{node_id}:stop`
Stops a running VM gracefully. ^[1]
- `POST /v1/compute/nodes/{node_id}:reboot`
Reboots a VM, preserving disks and configuration. ^[1]
- `DELETE /v1/compute/nodes/{node_id}`
Destroys a VM either synchronously or asynchronously, depending on parameters, and returns status or job tracking metadata. ^[1]

6. Storage APIs

The storage APIs (volumes and snapshots) are implemented as part of the Libcloud driver capabilities and exposed through dedicated endpoints. The summary file notes that volume/snapshot capabilities are included in the connection capability report; individual endpoints for create/list/delete are described in `ARCHITECTURE.md` but are not fully reproduced in `rest_api_summary.md`. ^[1]

At a high level:

- Volume endpoints cover listing, creating, attaching, detaching, and deleting block storage volumes for AWS and Nutanix where supported. ^[1]
- Snapshot endpoints cover creation and deletion of volume or VM snapshots, with provider-specific behavior abstracted behind Libcloud's snapshot interfaces. ^[1]

(If you want, I can pull the exact path/method matrix from `ARCHITECTURE.md` and expand this section to the same level of detail as compute.)

7. Network and Security APIs

Network-related endpoints manage floating IPs, key pairs, security groups, and firewalls, again based on Libcloud capabilities for the target provider. ^[1]

In summary:

- Floating IP endpoints allow allocation, listing, association, and release of IP addresses that can be attached to nodes. ^[1]
- Key pair endpoints support listing, creation, import, and deletion of SSH key pairs used for instance access. ^[1]
- Security group endpoints manage ingress/egress rules and group membership for nodes. ^[1]

- Firewall-related endpoints expose high-level rule and policy management abstractions where supported by Libcloud.^[1]

The exact REST paths and methods (e.g., GET `/v1/network/floating_ips`, POST `/v1/network/keypairs`, etc.) are detailed in the API Endpoint Tables section of `ARCHITECTURE.md`.^[1]

8. Miscellaneous and Operational APIs

The service includes additional endpoints that support health checking, metadata retrieval, and asynchronous job tracking.^[1]

Typical functions include:

- Health and ping endpoints to verify that the API server is reachable and that core subsystems are functioning.^[1]
- Metadata endpoints returning build, version, and configuration details for the deployment.^[1]
- Async job tracking endpoints allowing clients to query the status of long-running operations such as image creation or VM provisioning.^[1]

9. Security and Authorization Model

All operational endpoints that interact with providers require:

1. A valid Bearer token obtained via the Auth APIs.^[1]
2. A connection payload describing credentials and provider selection.^[1]

Before any cloud operation is performed, the API executes `policy_engine.authorize_connection()` to ensure that both the token's scope and the connection's attributes are permitted for the requested action. This design enforces a clear separation between identity (tokens), connectivity (driver configuration), and operations (actual Libcloud calls).^[1]

10. Summary

The `libcloud.rest` implementation provides a unified, stateless REST interface for multi-cloud operations across AWS and Nutanix, structured into authentication, discovery, connection, compute, storage, network, and operational endpoint groups. The design leverages Libcloud to normalize provider differences while relying on JWT-based authentication and explicit connection payloads for secure, auditable access.^[1]

Would you like me to extend this report by reading `ARCHITECTURE.md` and generating a full method/path table for volumes, snapshots, networking, and jobs to match the level of detail of the compute section?

1. rest_api_summary.md

2. ARCHITECTURE.md